MAPA AUTO-ORGANIZÁVEL DE KOHONEN COM TOPOLOGIA LINEAR

Resumo

Este trabalho resolve o Exercício 4.1 de Fausett (1994): uma rede de Kohonen com duas entradas e 50 unidades de cluster dispostas em linha é treinada com 100 pontos uniformemente distribuídos no disco $x_1^2 + x_2^2 < 0{,}25$, usando vizinhança fixa de raio $R = 1$ e taxa de aprendizado decrescendo linearmente de 0,5 a 0,01 em 100 épocas. A implementação foi feita em Python e NumPy, sem bibliotecas de SOM prontas. Em dez épocas o emaranhado aleatório inicial (209 cruzamentos da cadeia) é recolhido para dentro do disco e a cadeia de pesos passa a serpentear pela região dos dados; o erro de quantização médio cai de 0,140 para 0,042. Com $R = 1$ fixo, porém, restam alguns cruzamentos ("nós") que a rede não consegue desfazer. Análises complementares mostram que o tipo de decaimento de α pouco altera o resultado, que a quantidade de nós depende da inicialização e que um raio inicial grande elimina os nós por completo.

Palavras-chave: mapas auto-organizáveis; Kohonen; aprendizado competitivo; preservação topológica.

1 Introdução

Os mapas auto-organizáveis (*Self-Organizing Maps*, SOM), propostos por Kohonen (1982), são redes de aprendizado *competitivo* e *não supervisionado*. As unidades de saída (unidades de cluster) estão arranjadas segundo uma topologia fixa, como uma linha ou uma grade, e cada uma possui um vetor de pesos com a mesma dimensão das entradas. Quando um padrão $\mathbf{x}$ é apresentado, as unidades competem: vence aquela cujo vetor de pesos está mais próximo de $\mathbf{x}$. Diferentemente do aprendizado competitivo puro ("o vencedor leva tudo"), no SOM o vencedor e seus *vizinhos topológicos* são deslocados na direção do padrão (FAUSETT, 1994; HAYKIN, 2001).

Essa cooperação local é o mecanismo da *preservação topológica*: unidades vizinhas na grade de saída são repetidamente arrastadas para as mesmas regiões do espaço de entrada e, ao fim do treino, passam a representar regiões vizinhas. O mapa resultante é uma projeção ordenada do espaço de entrada sobre a topologia de saída (KOHONEN, 2001). Quando a topologia é linear e a entrada é bidimensional, a rede precisa "costurar" uma curva unidimensional pela região dos dados, algo próximo de uma curva que preenche o plano. Essa é a situação do Exercício 4.1, e permite visualizar diretamente, no plano (w_1, w_2), o processo de auto-organização.

2 Metodologia

2.1 Arquitetura e algoritmo

A rede tem $n = 2$ unidades de entrada e $m = 50$ unidades de cluster $C_1, \ldots, C_{50}$ em topologia linear. O vetor de pesos da unidade C_j é $\mathbf{w}_{\cdot j} = (w_{1j}, w_{2j})$, inicializado com componentes independentes $w_{ij} \sim U(-1, 1)$. Para cada padrão $\mathbf{x}$:

1. **Competição:** calcula-se $D(j) = \sum_{i=1}^{2} (w_{ij} - x_i)^2$ para $j = 1, \ldots, 50$, e o vencedor é $J = \arg\min_j D(j)$.

2. **Cooperação:** a vizinhança de raio $R = 1$ é $\mathcal{N}(J) = \{J-1, J, J+1\} \cap \{1, \ldots, 50\}$; nas bordas o vizinho inexistente é simplesmente ignorado, de modo que C_1 e C_{50} têm um único vizinho.

3. **Adaptação:** para todo $j \in \mathcal{N}(J)$,

$$w_{ij}(\text{novo}) = w_{ij}(\text{velho}) + \alpha \left[x_i - w_{ij}(\text{velho}) \right]. \tag{1}$$

Uma época consiste na apresentação dos 100 padrões em ordem aleatória, sorteada novamente a cada época. O raio $R = 1$ permanece fixo durante todo o treino, e o treinamento dura 100 épocas.

2.2 Geração dos dados

Os 100 pontos de treino foram obtidos por amostragem por rejeição: sorteiam-se $x_1, x_2 \sim U(-0{,}5; 0{,}5)$ e o ponto é aceito somente se $x_1^2 + x_2^2 < 0{,}25$. O processo se repete até haver 100 pontos aceitos, o que resulta em uma distribuição uniforme no disco de raio 0,5. A taxa de aceitação esperada é a razão entre as áreas do disco e do quadrado, $\pi/4 \approx 0{,}785$; foram necessários 122 sorteios (taxa observada de 0,820). Os pontos estão em `dados_treino.csv`.

Foram usados dois geradores pseudoaleatórios independentes: um para os dados (semente 2026) e outro para a rede, que define os pesos iniciais e a ordem de apresentação (semente 42). Com um único gerador, os primeiros pesos sorteados em $U(-1, 1)$ seriam cópias escaladas por 2 dos primeiros candidatos a ponto de treino, o que criaria uma correlação artificial entre inicialização e dados.

2.3 Decaimento da taxa de aprendizado

O enunciado pede que α comece em 0,5 e seja reduzido gradualmente até 0,01 ao longo de 100 épocas, sem fixar a forma da redução. Adotou-se o **decaimento linear por época**:

$$\alpha(t) = \alpha_0 + (\alpha_f - \alpha_0) \frac{t}{T-1}, \qquad t = 0, 1, \ldots, T-1, \tag{2}$$

com $\alpha_0 = 0{,}5$, $\alpha_f = 0{,}01$ e $T = 100$. O valor é constante dentro de cada época, e a primeira e a última época usam exatamente 0,5 e 0,01. Essa é a leitura mais literal de "reduzir gradualmente" e, além disso, mantém α relativamente alto por mais tempo, o que favorece a fase de ordenação. A função de decaimento é um parâmetro do treino. Para comparação, também foi implementado o decaimento exponencial (geométrico) entre os mesmos extremos, $\alpha(t) = \alpha_0\,(\alpha_f/\alpha_0)^{t/(T-1)}$. As duas curvas aparecem na Figura 1.

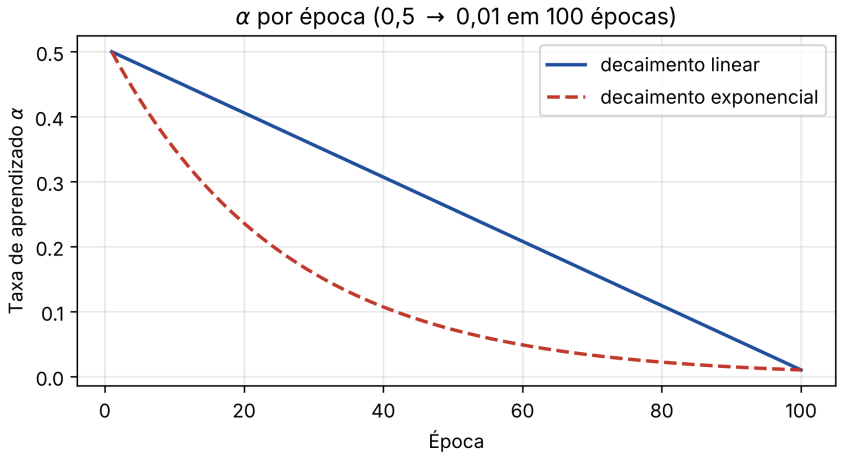


Figura 1 – Taxa de aprendizado em cada época para os decaimentos linear (padrão) e exponencial.

2.4 Métricas de acompanhamento

Além da inspeção visual, três métricas foram calculadas:

- **Erro de quantização médio**: $\mathrm{EQ} = \frac{1}{P}\sum_{p=1}^{P} \min_j \|\mathbf{x}_p - \mathbf{w}_{\cdot j}\|$, a distância média de cada ponto ao vetor de pesos vencedor;
- **Número de cruzamentos**: quantidade de pares de segmentos não adjacentes da poligonal C_1–C_2–$\cdots$–C_{50} que se intersectam. Zero cruzamentos indica uma curva sem "nós" no plano;
- **Unidades mortas**: unidades que não vencem para nenhum padrão de treino.

A implementação (`som_kohonen.py`, Apêndice A) usa apenas Python e NumPy no algoritmo e Matplotlib nas figuras. O mesmo conteúdo está no notebook `som_kohonen.ipynb`.

3 Resultados

A Tabela 1 reproduz a saída do programa a cada 10 épocas, e a Figura 2 mostra a configuração dos 50 vetores de peso nesses mesmos instantes, sobrepostos aos pontos de treino e ao círculo $x_1^2 + x_2^2 = 0{,}25$.

Tabela 1 – Evolução do treino principal ($R = 1$, decaimento linear, semente 42). α é o valor usado na época indicada.

Época	α	EQ	Cruzamentos
0	–	0,1397	209
10	0,4555	0,0485	15
20	0,4060	0,0466	4
30	0,3565	0,0464	4
40	0,3070	0,0453	4
50	0,2575	0,0447	4
60	0,2080	0,0418	4
70	0,1585	0,0414	4
80	0,1090	0,0416	4
90	0,0595	0,0420	4
100	0,0100	0,0417	4

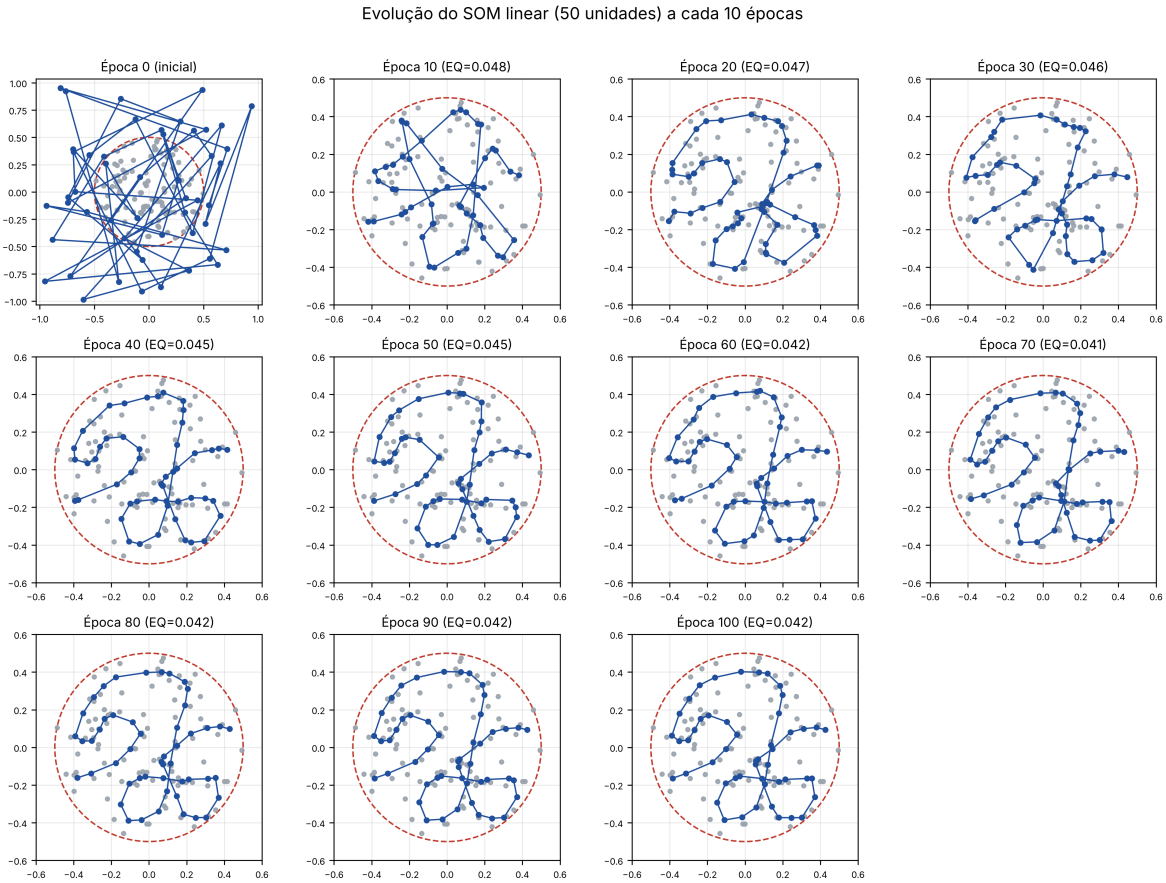
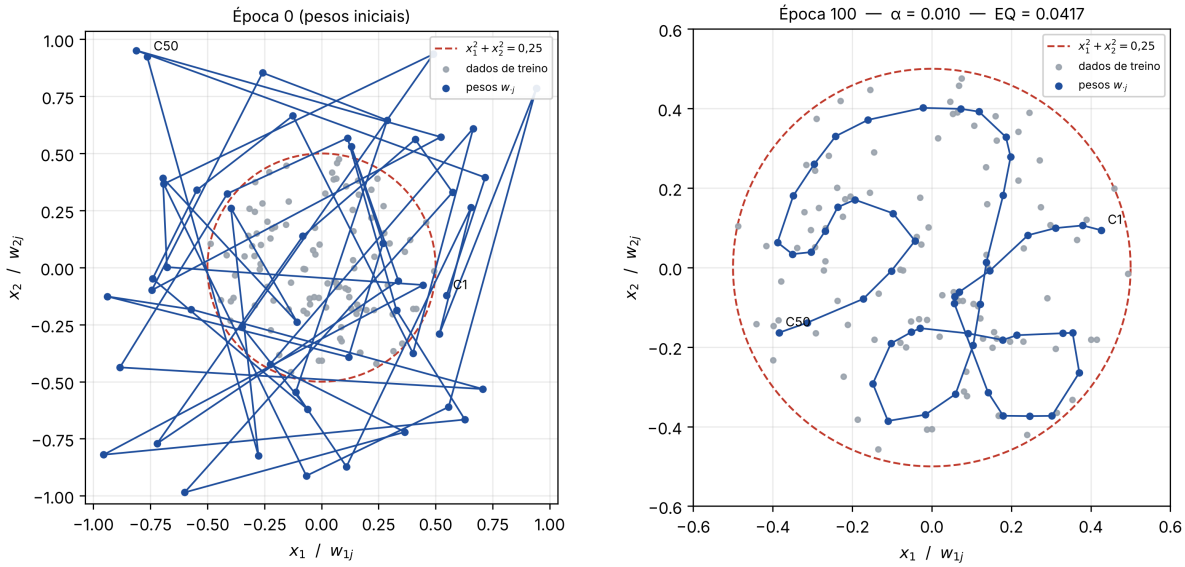


Figura 2 – Vetores de peso das 50 unidades (pontos azuis ligados na ordem C_1–C_{50}) nas épocas 0, 10, …, 100, sobre os dados de treino (cinza) e o círculo $x_1^2 + x_2^2 = 0{,}25$ (tracejado). As figuras individuais de cada época estão em `figuras/som_epoca_XXX.png`.



(a) Época 0: pesos iniciais em $U(-1,1)^2$.

(b) Época 100: configuração final.

Figura 3 – Estado inicial e final da rede, em detalhe.

A evolução tem duas fases nítidas:

- **Ordenação (épocas 1–10).** Na época 0 os pesos estão espalhados por todo o quadrado $[-1,1]^2$. Como a maioria fica fora do disco, a cadeia atravessa o plano 209 vezes. Com $\alpha \approx 0{,}5$, cada atualização leva o vencedor e seus dois vizinhos a meio caminho do padrão apresentado, e em dez épocas todos os pesos foram puxados para dentro do disco. A maior parte do emaranhado se desfaz (de 209 para 15 cruzamentos), e o erro de quantização cai quase três vezes, de 0,140 para 0,048.

- **Convergência (épocas 10–100).** A forma global da curva praticamente congela. Os cruzamentos caem para 4 até a época 20 e permanecem nesse valor até o fim. O que muda é o detalhe local: com α decrescendo, os pesos se acomodam nos agrupamentos de pontos, os segmentos ficam mais regulares e o EQ desce lentamente até 0,042 (Figura 4).

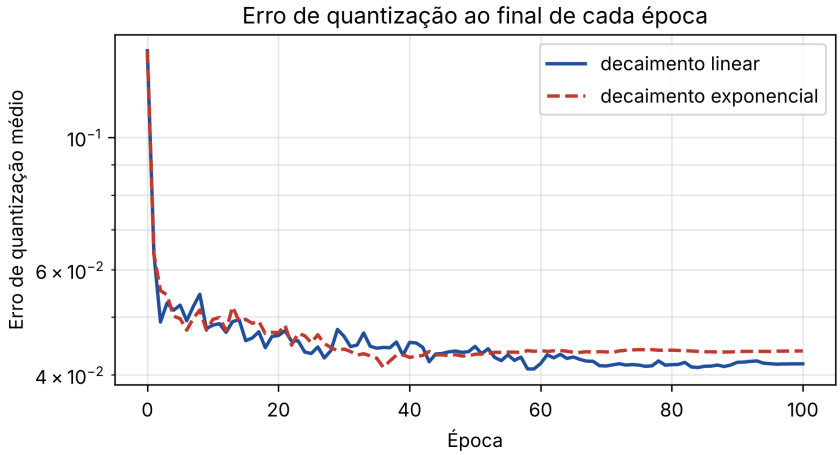


Figura 4 – Erro de quantização médio ao final de cada época (escala logarítmica) para os dois decaimentos de α.

Ao final, a cadeia tem comprimento total 4,31 e espaçamento médio de 0,088 entre unidades consecutivas (mínimo de 0,016, máximo de 0,239). Ela serpenteia por todo o disco, alcançando raio 0,48, mas mantém 4 cruzamentos concentrados próximos ao centro e ao quadrante inferior direito (Figura 3b). As unidades extremas C_1 e C_{50} ficam junto à borda do disco.

3.1 Análises complementares

Decaimento exponencial. Com a mesma semente, o decaimento exponencial leva a uma configuração final de forma semelhante, com 3 cruzamentos e EQ = 0,0439 (Figura 5). Como α cai mais depressa (já vale 0,11 na época 40), o mapa "congela" mais cedo: o EQ quase não muda depois da época 40 (Figura 4).

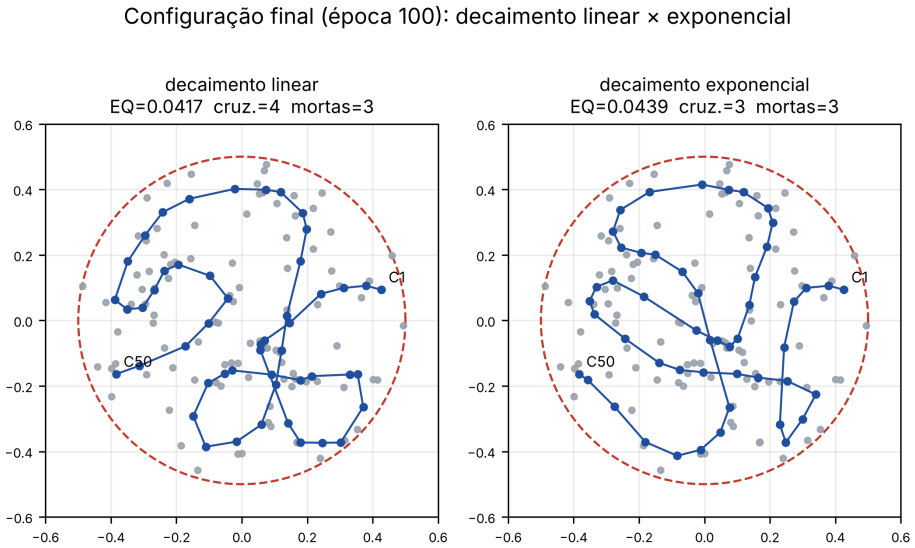


Figura 5 – Configuração final com decaimento linear e exponencial de α (mesmos dados e mesma semente).

Sementes diferentes. Mantidos os dados e o decaimento linear, variou-se a semente da rede, que define os pesos iniciais e a ordem de apresentação. A Tabela 2 e a Figura 6 mostram que o EQ final é estável, entre 0,040 e 0,043, mas a quantidade de nós residuais varia de 0 a 4. A semente 1 produziu uma curva sem cruzamentos.

Tabela 2 – Resultado final (época 100) para diferentes sementes da rede.

Semente	EQ	Cruzamentos	Unidades mortas	Comprimento da cadeia
42	0,0417	4	3	4,31
1	0,0426	0	4	3,88
7	0,0431	4	2	4,57
2024	0,0397	2	1	4,35
31415	0,0421	2	2	4,11

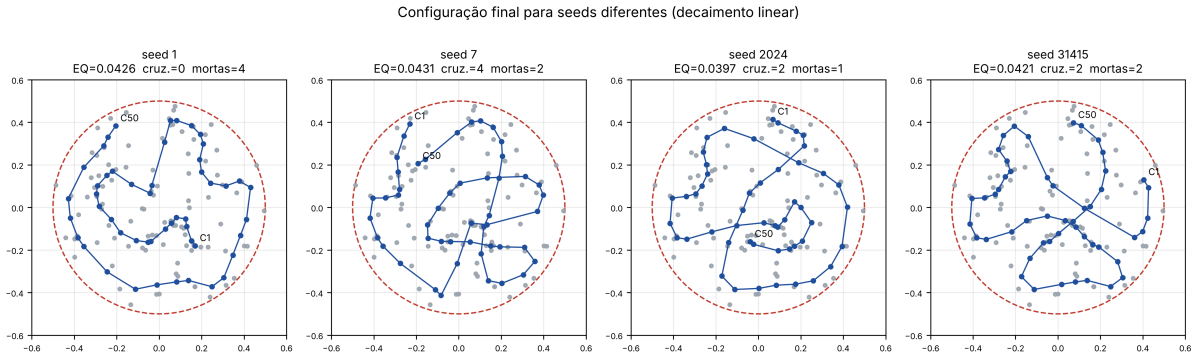


Figura 6 – Configuração final para quatro sementes adicionais.

Raio decrescente (fora do enunciado). Para isolar o papel da vizinhança, repetiu-se o treino com a mesma semente, mas com raio inicial $R = 10$, reduzido linearmente até $R = 1$ na época 50 e mantido em 1 depois disso. A cadeia fica sem nenhum cruzamento já na época 10 e termina desembaraçada, com EQ $= 0{,}0401$ e apenas uma unidade morta (Figura 7). O preço é um EQ maior durante a fase de raio grande (0,108 na época 10, contra 0,048 com $R = 1$), porque cada atualização arrasta muitas unidades ao mesmo tempo e a cadeia fica "tensionada".

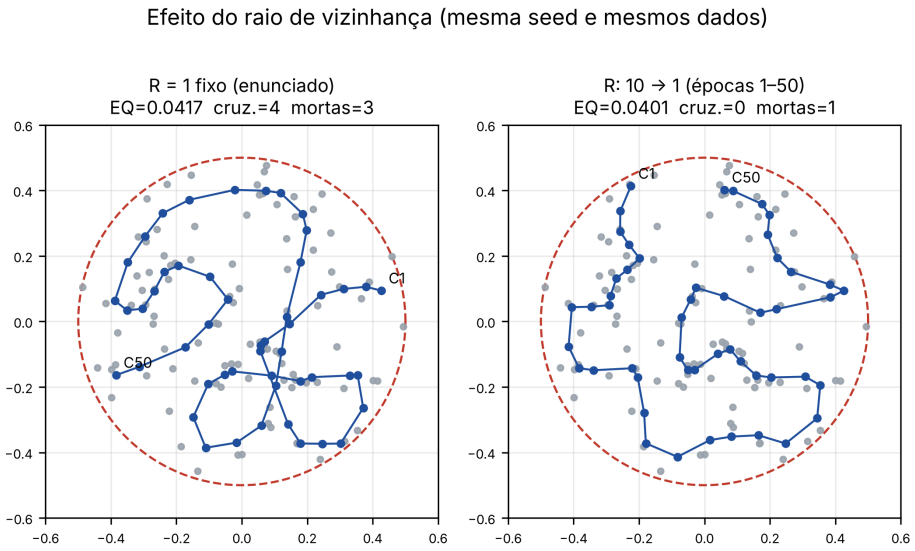


Figura 7 – Efeito do raio de vizinhança: $R = 1$ fixo (enunciado) e R decrescente de 10 a 1 (mesmos dados e mesma semente).

4 Discussão

Efeito da vizinhança $R = 1$. A vizinhança é o que transforma o aprendizado competitivo em mapa topológico. Com $R = 1$, cada atualização acopla apenas três unidades, então a ordem da cadeia só se propaga por contato entre vizinhos imediatos. Isso basta para desfazer cruzamentos *locais*. Um "nó" formado entre trechos distantes da cadeia, porém, exigiria que dezenas de unidades se movessem de forma coordenada, e nenhuma atualização isolada produz esse movimento. Na prática, a configuração global fica decidida nas primeiras épocas e depende da inicialização (Tabela 2). O experimento com raio decrescente confirma o diagnóstico. É por isso que Kohonen (2001) e Haykin (2001) recomendam começar com uma vizinhança que abranja boa parte da rede e reduzi-la gradualmente. O exercício, ao fixar $R = 1$, evidencia justamente o que se perde sem essa fase.

Efeito do decaimento de α. Um α alto no início permite grandes deslocamentos e acelera a ordenação. Um α pequeno no fim faz cada peso convergir para uma média local dos padrões que ele vence, o que reduz o EQ e estabiliza o mapa. Os dois decaimentos testados terminam em configurações equivalentes. O linear mantém a rede plástica por mais tempo, e o exponencial congela o mapa antes. Com $R = 1$ fixo, porém, mais plasticidade não basta para desfazer os nós globais. A ordenação depende mais do raio do que de α.

Densidade de unidades e densidade dos dados. Há 50 unidades para 100 pontos, cerca de dois pontos por unidade. Na rede principal, 13 unidades vencem um único ponto, 19 vencem dois, 12 vencem três e o máximo é cinco. Três unidades não vencem nenhum ponto. Unidades

mortas como essas não são desperdício puro: ficam entre unidades ativas e funcionam como "elos" de interpolação: a restrição topológica obriga a curva a passar por ali. A distribuição radial dos pesos acompanha a dos dados. A fração de pesos com $\|\mathbf{w}\| < 0{,}25$ é 0,30, contra 0,31 dos pontos, e os raios médios são 0,298 e 0,306. Isso indica que a rede aloca unidades aproximadamente na proporção da densidade, como esperado para um SOM (KOHONEN, 2001). Com apenas 100 amostras, a densidade "vista" pela rede não é uniforme, e flutuações locais (aglomerados e vazios) explicam a variação no espaçamento entre unidades.

Dobras e nós residuais. Para preencher um disco, uma curva unidimensional precisa se dobrar, e as dobras em "U" observadas são inerentes ao problema, não defeitos. Defeitos são os cruzamentos, em que dois trechos distantes da cadeia representam regiões sobrepostas. Nesses pontos a preservação topológica é violada localmente: unidades distantes na cadeia ficam próximas no plano. Também se observa o efeito de borda: as unidades extremas têm um só vizinho, são menos "puxadas" e tendem a se fixar na periferia do disco.

5 Conclusão

A rede de Kohonen linear do Exercício 4.1 foi implementada e treinada conforme o enunciado. Os resultados mostram o comportamento esperado de um SOM. Em cerca de dez épocas o emaranhado aleatório se recolhe para o disco e se ordena em uma curva que serpenteia pela região dos dados. Depois disso, a redução de α refina a quantização, levando o EQ de 0,140 a 0,042. A distribuição das unidades acompanha a dos dados.

O raio fixo $R = 1$ limita a ordenação global. A maioria das execuções termina com alguns nós residuais, cuja quantidade depende da inicialização. O decaimento de α, linear ou exponencial, tem efeito secundário, enquanto uma vizinhança inicial ampla elimina os nós por completo. Esses resultados reforçam o papel central da cooperação entre vizinhos na auto-organização. Uma extensão natural seria medir a frequência de nós em muitas sementes e avaliar métricas formais de preservação topológica, como o erro topográfico.

Referências

FAUSETT, L. **Fundamentals of neural networks**: architectures, algorithms, and applications. Englewood Cliffs: Prentice Hall, 1994.

HAYKIN, S. **Redes neurais**: princípios e prática. 2. ed. Porto Alegre: Bookman, 2001.

KOHONEN, T. Self-organized formation of topologically correct feature maps. **Biological Cybernetics**, v. 43, n. 1, p. 59–69, 1982.

KOHONEN, T. **Self-organizing maps**. 3. ed. Berlin: Springer, 2001.

A Código-fonte

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Trabalho 09 - Mapas Auto-Organizaveis de Kohonen (SOM)
Exercicio 4.1 - Fausett, "Fundamentals of Neural Networks" (1994)

Disciplina : EL056 - Redes Neurais Artificiais (PPGEELT/UFU)
Professor : Prof. Dr. Keiji Yamanaka
Aluno : Lucas Albino Martins (matricula 12622EEL006)

Descricao
---------
Rede de Kohonen com 2 unidades de entrada e 50 unidades de cluster dispostas
em topologia LINEAR (1D). O vencedor J e seus vizinhos imediatos (J-1, J+1)
aprendem (raio R = 1 fixo). A taxa de aprendizado decai de 0,5 ate 0,01 ao
longo de 100 epocas. Os dados de treino sao 100 pontos uniformes dentro do
disco x1^2 + x2^2 < 0,25 (raio 0,5), obtidos por amostragem por rejeicao.

Implementacao em Python puro + NumPy (nenhuma biblioteca de SOM pronta).

Uso
---
    python som_kohonen.py # experimento principal + extras
    python som_kohonen.py --sem-extras # apenas o experimento principal
    python som_kohonen.py --decaimento exponencial --seed 7

Saidas (pasta definida em --saida, padrao: diretorio atual)
------
    dados_treino.csv
    figuras/som_epoca_000.png ... som_epoca_100.png
    figuras/som_painel.png
    figuras/alpha_por_epoca.png
    figuras/erro_quantizacao.png
    figuras/comparacao_decaimento.png (extra)
    figuras/comparacao_seeds.png (extra)
    figuras/comparacao_raio.png (extra, fora do enunciado)
    pesos_finais.csv
"""

from __future__ import annotations

import argparse
import csv
import os
from typing import Callable, Dict, List

import numpy as np
import matplotlib

matplotlib.use("Agg") # renderizacao sem janela (gera apenas arquivos)
import matplotlib.pyplot as plt # noqa: E402

# ============================================================================
# 1. Parametros do enunciado
# ============================================================================
N_ENTRADAS = 2 # dimensao do vetor de entrada (x1, x2)
N_CLUSTERS = 50 # unidades de cluster na topologia linear
```

```
RAIO_VIZINHANCA = 1 # R = 1 (fixo): vencedor J, J-1 e J+1
ALFA_INICIAL = 0.5
ALFA_FINAL = 0.01
N_EPOCAS = 100
N_PONTOS = 100
RAIO_DISCO = 0.5 # x1^2 + x2^2 < 0,25 <=> raio 0,5
SEED_PADRAO = 42 # seed da rede (pesos iniciais + ordem dos padroes)
SEED_DADOS = 2026 # seed independente para os dados de treino
EPOCAS_PLOT = list(range(0, N_EPOCAS + 1, 10)) # 0, 10, 20, ..., 100


# ============================================================================
# 2. Funcoes de decaimento da taxa de aprendizado
# ============================================================================
# Cada funcao recebe o indice da epoca (0 = primeira epoca de treino) e o
# numero total de epocas, e devolve o alfa usado durante aquela epoca.
# A primeira epoca usa exatamente ALFA_INICIAL e a ultima exatamente
# ALFA_FINAL, de modo que "0,5 reduzido gradualmente ate 0,01 em 100 epocas"
# seja cumprido literalmente.

def decaimento_linear(epoca: int, n_epocas: int,
                      a0: float = ALFA_INICIAL, af: float = ALFA_FINAL) -> float:
    """alfa(t) = a0 + (af - a0) * t / (T - 1), t = 0, ..., T-1."""
    if n_epocas <= 1:
        return a0
    return a0 + (af - a0) * epoca / (n_epocas - 1)


def decaimento_exponencial(epoca: int, n_epocas: int,
                           a0: float = ALFA_INICIAL, af: float = ALFA_FINAL) -> float:
    """alfa(t) = a0 * (af / a0) ** (t / (T - 1)) (geometrico entre a0 e af)."""
    if n_epocas <= 1:
        return a0
    return a0 * (af / a0) ** (epoca / (n_epocas - 1))


DECAIMENTOS: Dict[str, Callable[[int, int], float]] = {
    "linear": decaimento_linear,
    "exponencial": decaimento_exponencial,
}


# ============================================================================
# 3. Geracao dos dados de treino (amostragem por rejeicao)
# ============================================================================

def gerar_dados_disco(n_pontos: int, rng: np.random.Generator,
                      raio: float = RAIO_DISCO) -> np.ndarray:
    """Sorteia x1, x2 ~ U(-raio, raio) e aceita o ponto somente se
    x1^2 + x2^2 < raio^2, repetindo ate obter n_pontos pontos aceitos."""
    pontos: List[np.ndarray] = []
    n_sorteios = 0
    while len(pontos) < n_pontos:
        candidato = rng.uniform(-raio, raio, size=2)
        n_sorteios += 1
        if candidato[0] ** 2 + candidato[1] ** 2 < raio ** 2:
            pontos.append(candidato)
    taxa = n_pontos / n_sorteios
    print(f"[dados] {n_pontos} pontos aceitos em {n_sorteios} sorteios "
```

```
        f"(taxa de aceitacao = {taxa:.3f}; teorico pi/4 = {np.pi/4:.3f})")
    return np.array(pontos)


def salvar_csv(caminho: str, dados: np.ndarray) -> None:
    with open(caminho, "w", newline="", encoding="utf-8") as f:
        escritor = csv.writer(f)
        escritor.writerow(["indice", "x1", "x2"])
        for i, (x1, x2) in enumerate(dados, start=1):
            escritor.writerow([i, f"{x1:.10f}", f"{x2:.10f}"])


def carregar_csv(caminho: str) -> np.ndarray:
    return np.loadtxt(caminho, delimiter=",", skiprows=1, usecols=(1, 2))


# ============================================================================
# 4. A rede de Kohonen com topologia linear
# ============================================================================

class SOMLinear:
    """Mapa auto-organizavel de Kohonen com unidades de cluster em linha.

    Os pesos ficam em uma matriz `pesos` de forma (n_clusters, n_entradas):
    a linha j e o vetor de pesos w_.j da unidade de cluster C_{j+1}
    (indexacao 0 no codigo, 1 no texto).
    """

    def __init__(self, n_entradas: int = N_ENTRADAS, n_clusters: int = N_CLUSTERS,
                 raio: int = RAIO_VIZINHANCA, rng: np.random.Generator | None = None):
        self.n_entradas = n_entradas
        self.n_clusters = n_clusters
        self.raio = raio
        self.rng = rng if rng is not None else np.random.default_rng(SEED_PADRAO)
        # Pesos iniciais: cada componente ~ U(-1, 1)
        self.pesos = self.rng.uniform(-1.0, 1.0, size=(n_clusters, n_entradas))

    # ---- competicao ----------------------------------------------------------
    def distancias_quadradas(self, x: np.ndarray) -> np.ndarray:
        """D(j) = sum_i (w_ij - x_i)^2 para todas as unidades j."""
        return np.sum((self.pesos - x) ** 2, axis=1)

    def vencedor(self, x: np.ndarray) -> int:
        """Indice J da unidade com menor distancia euclidiana ao quadrado."""
        return int(np.argmin(self.distancias_quadradas(x)))

    # ---- cooperacao ----------------------------------------------------------
    def vizinhanca(self, J: int) -> range:
        """Unidades J-R, ..., J+R, descartando as que caem fora de [0, n-1]
        (nas bordas o vizinho inexistente e simplesmente ignorado)."""
        inicio = max(0, J - self.raio)
        fim = min(self.n_clusters - 1, J + self.raio)
        return range(inicio, fim + 1)

    # ---- adaptacao -----------------------------------------------------------
    def atualizar(self, x: np.ndarray, alfa: float) -> int:
        """w_ij(new) = w_ij(old) + alfa * (x_i - w_ij(old)) para j na vizinhanca."""
        J = self.vencedor(x)
        for j in self.vizinhanca(J):
```

```
176             self.pesos[j] += alfa * (x - self.pesos[j])
177         return J
178
179     def treinar_epoca(self, dados: np.ndarray, alfa: float) -> None:
180         """Uma epoca: todos os padroes, em ordem aleatoria (nova a cada epoca)."""
181         for indice in self.rng.permutation(len(dados)):
182             self.atualizar(dados[indice], alfa)
183
184     # ---- metricas ------------------------------------------------------------
185     def erro_quantizacao(self, dados: np.ndarray) -> float:
186         """Distancia euclidiana media de cada ponto ao vetor de pesos vencedor."""
187         d2 = np.sum((dados[:, None, :] - self.pesos[None, :, :]) ** 2, axis=2)
188         return float(np.mean(np.sqrt(d2.min(axis=1))))
189
190     def comprimento_cadeia(self) -> float:
191         """Soma dos comprimentos dos segmentos C1-C2-...-C50."""
192         return float(np.sum(np.linalg.norm(np.diff(self.pesos, axis=0), axis=1)))
193
194     def n_cruzamentos(self) -> int:
195         """Numero de pares de segmentos nao adjacentes da cadeia que se cruzam.
196         Zero cruzamentos = curva sem "nos" (preservacao topologica no plano)."""
197         P = self.pesos
198         n_seg = len(P) - 1
199
200         def orient(a, b, c):
201             return np.sign((b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0]))
202
203         total = 0
204         for i in range(n_seg):
205             for k in range(i + 2, n_seg):
206                 a, b, c, d = P[i], P[i + 1], P[k], P[k + 1]
207                 if (orient(a, b, c) * orient(a, b, d) < 0 and
208                         orient(c, d, a) * orient(c, d, b) < 0):
209                     total += 1
210         return total
211
212     def unidades_mortas(self, dados: np.ndarray) -> int:
213         """Unidades que nao vencem para nenhum padrao de treino."""
214         d2 = np.sum((dados[:, None, :] - self.pesos[None, :, :]) ** 2, axis=2)
215         return self.n_clusters - len(np.unique(d2.argmin(axis=1)))
216
217
218 # ============================================================================
219 # 5. Laco de treino com registro de historico
220 # ============================================================================
221
222 def raio_decrescente(epoca: int, n_epocas: int, r0: int = 10, epoca_r1: int = 50) -> int:
223     """(Somente analise extra, fora do enunciado) raio que cai linearmente de
224     r0 ate 1 nas primeiras `epoca_r1` epocas e fica em 1 dai em diante."""
225     return max(1, int(round(r0 - (r0 - 1) * epoca / epoca_r1)))
226
227
228 def treinar(dados: np.ndarray, seed: int = SEED_PADRAO, n_epocas: int = N_EPOCAS,
229             decaimento: Callable[[int, int], float] = decaimento_linear,
230             epocas_registro: List[int] | None = None, verbose: bool = True,
231             raio_fn: Callable[[int, int], int] | None = None) -> dict:
232     """Treina um SOMLinear e devolve um dicionario com:
233     - 'som' : a rede treinada
234     - 'snapshots' : {epoca: copia dos pesos ao FINAL daquela epoca}
```

```
                    (epoca 0 = pesos iniciais, antes do treino)
    - 'alfas' : alfa usado em cada epoca 1..n_epocas
    - 'erros' : erro de quantizacao ao final de cada epoca 0..n_epocas
    Se `raio_fn` for dado, o raio de vizinhanca passa a variar por epoca
    (usado apenas na analise extra; o enunciado exige R = 1 fixo).
    """
    if epocas_registro is None:
        epocas_registro = list(range(0, n_epocas + 1, 10))
    rng = np.random.default_rng(seed)
    som = SOMLinear(rng=rng)

    snapshots = {0: som.pesos.copy()}
    alfas: List[float] = []
    erros = [som.erro_quantizacao(dados)]

    if verbose:
        print(f"{'epoca':>6} | {'alfa':>8} | {'erro quant.':>11} | {'cruzam.':>7}")
        print("-" * 44)
        print(f"{0:>6} | {'-':>8} | {erros[0]:>11.5f} | {som.n_cruzamentos():>7}")

    for epoca in range(1, n_epocas + 1):
        alfa = decaimento(epoca - 1, n_epocas)
        if raio_fn is not None:
            som.raio = raio_fn(epoca - 1, n_epocas)
        som.treinar_epoca(dados, alfa)
        alfas.append(alfa)
        erros.append(som.erro_quantizacao(dados))
        if epoca in epocas_registro:
            snapshots[epoca] = som.pesos.copy()
            if verbose:
                print(f"{epoca:>6} | {alfa:>8.4f} | {erros[-1]:>11.5f} | "
                      f"{som.n_cruzamentos():>7}")

    return {"som": som, "snapshots": snapshots, "alfas": np.array(alfas),
            "erros": np.array(erros)}


# ============================================================================
# 6. Visualizacao
# ============================================================================
COR_DADOS = "#9aa3ad"
COR_REDE = "#1f4e9c"
COR_CIRCULO = "#c0392b"


def desenhar_som(ax, pesos: np.ndarray, dados: np.ndarray, titulo: str = "",
                 limite: float | None = None, rotular_extremos: bool = True) -> None:
    """Pontos de treino + circulo x1^2+x2^2=0,25 + cadeia C1-C2-...-C50.
    Se `limite` for None, o enquadramento se ajusta aos pesos (no minimo
    [-0,6; 0,6], que mostra o disco inteiro; na epoca 0 abre para [-1; 1])."""
    if limite is None:
        limite = max(0.6, float(np.abs(pesos).max()) + 0.05)
    theta = np.linspace(0, 2 * np.pi, 400)
    ax.plot(RAIO_DISCO * np.cos(theta), RAIO_DISCO * np.sin(theta),
            color=COR_CIRCULO, lw=1.2, ls="--", label=r"$x_1^2+x_2^2=0{,}25$")
    ax.scatter(dados[:, 0], dados[:, 1], s=10, color=COR_DADOS, alpha=0.9,
               label="dados de treino", zorder=2)
    ax.plot(pesos[:, 0], pesos[:, 1], "-", color=COR_REDE, lw=1.1, zorder=3)
    ax.scatter(pesos[:, 0], pesos[:, 1], s=14, color=COR_REDE, zorder=4,
```

```
294                 label="pesos $w_{\\cdot j}$")
295     if rotular_extremos:
296         for idx, nome in ((0, "C1"), (len(pesos) - 1, f"C{len(pesos)}")):
297             ax.annotate(nome, pesos[idx], xytext=(4, 4), textcoords="offset points",
298                         fontsize=8, color="black", zorder=5)
299     ax.set_xlim(-limite, limite)
300     ax.set_ylim(-limite, limite)
301     ax.set_aspect("equal")
302     ax.set_xlabel("$x_1$ / $w_{1j}$")
303     ax.set_ylabel("$x_2$ / $w_{2j}$")
304     ax.grid(alpha=0.25)
305     if titulo:
306         ax.set_title(titulo, fontsize=10)
307
308
309 def salvar_figuras_epocas(snapshots: dict, dados: np.ndarray, pasta: str,
310                           alfas: np.ndarray, erros: np.ndarray, dpi: int = 300) -> None:
311     for epoca, pesos in sorted(snapshots.items()):
312         fig, ax = plt.subplots(figsize=(5.2, 5.2))
313         if epoca == 0:
314             titulo = "Época 0 (pesos iniciais)"
315         else:
316             titulo = (f"Época {epoca} \u2014 \u03b1 = {alfas[epoca-1]:.3f} \u2014 "
317                       f"EQ = {erros[epoca]:.4f}")
318         desenhar_som(ax, pesos, dados, titulo)
319         ax.legend(loc="upper right", fontsize=7, framealpha=0.9)
320         fig.tight_layout()
321         fig.savefig(os.path.join(pasta, f"som_epoca_{epoca:03d}.png"), dpi=dpi)
322         plt.close(fig)
323
324
325 def salvar_painel(snapshots: dict, dados: np.ndarray, caminho: str,
326                   erros: np.ndarray, dpi: int = 300) -> None:
327     epocas = sorted(snapshots)
328     n_col = 4
329     n_lin = int(np.ceil(len(epocas) / n_col))
330     fig, eixos = plt.subplots(n_lin, n_col, figsize=(3.3 * n_col, 3.3 * n_lin))
331     eixos = np.atleast_1d(eixos).ravel()
332     for ax, epoca in zip(eixos, epocas):
333         rot = "Época 0 (inicial)" if epoca == 0 else f"Época {epoca} (EQ={erros[epoca]:.3f})"
334         desenhar_som(ax, snapshots[epoca], dados, rot, rotular_extremos=False)
335         ax.set_xlabel("")
336         ax.set_ylabel("")
337         ax.tick_params(labelsize=7)
338     for ax in eixos[len(epocas):]:
339         ax.axis("off")
340     fig.suptitle("Evolução do SOM linear (50 unidades) a cada 10 épocas", fontsize=13)
341     fig.tight_layout(rect=(0, 0, 1, 0.97))
342     fig.savefig(caminho, dpi=dpi)
343     plt.close(fig)
344
345
346 def salvar_grafico_alfa(alfas_por_nome: Dict[str, np.ndarray], caminho: str,
347                         dpi: int = 300) -> None:
348     fig, ax = plt.subplots(figsize=(6.4, 3.6))
349     estilos = {"linear": ("-", COR_REDE), "exponencial": ("--", COR_CIRCULO)}
350     for nome, alfas in alfas_por_nome.items():
351         ls, cor = estilos.get(nome, ("-", "k"))
352         ax.plot(np.arange(1, len(alfas) + 1), alfas, ls, color=cor, lw=1.8,
```

```
353                 label=f"decaimento {nome}")
354     ax.set_xlabel("Época")
355     ax.set_ylabel(r"Taxa de aprendizado $\alpha$")
356     ax.set_title(r"$\alpha$ por época (0,5 $\rightarrow$ 0,01 em 100 épocas)")
357     ax.grid(alpha=0.3)
358     ax.legend()
359     fig.tight_layout()
360     fig.savefig(caminho, dpi=dpi)
361     plt.close(fig)
362
363
364 def salvar_grafico_erro(erros_por_nome: Dict[str, np.ndarray], caminho: str,
365                         dpi: int = 300) -> None:
366     fig, ax = plt.subplots(figsize=(6.4, 3.6))
367     estilos = {"linear": ("-", COR_REDE), "exponencial": ("--", COR_CIRCULO)}
368     for nome, erros in erros_por_nome.items():
369         ls, cor = estilos.get(nome, ("-", "k"))
370         ax.plot(np.arange(len(erros)), erros, ls, color=cor, lw=1.8,
371                 label=f"decaimento {nome}")
372     ax.set_yscale("log")
373     ax.set_xlabel("Época")
374     ax.set_ylabel("Erro de quantização médio")
375     ax.set_title("Erro de quantização ao final de cada época")
376     ax.grid(alpha=0.3, which="both")
377     ax.legend()
378     fig.tight_layout()
379     fig.savefig(caminho, dpi=dpi)
380     plt.close(fig)
381
382
383 def salvar_comparacao(resultados: Dict[str, dict], dados: np.ndarray, caminho: str,
384                       titulo: str, dpi: int = 300) -> None:
385     n = len(resultados)
386     fig, eixos = plt.subplots(1, n, figsize=(3.6 * n, 4.4))
387     eixos = np.atleast_1d(eixos)
388     for ax, (nome, res) in zip(eixos, resultados.items()):
389         som = res["som"]
390         rot = (f"{nome}\nEQ={res['erros'][-1]:.4f} "
391                f"cruz.={som.n_cruzamentos()} mortas={som.unidades_mortas(dados)}")
392         desenhar_som(ax, som.pesos, dados, rot, rotular_extremos=True)
393         ax.set_xlabel("")
394         ax.set_ylabel("")
395         ax.tick_params(labelsize=7)
396     fig.suptitle(titulo, fontsize=12)
397     fig.tight_layout(rect=(0, 0, 1, 0.94))
398     fig.savefig(caminho, dpi=dpi)
399     plt.close(fig)
400
401
402 # ============================================================================
403 # 7. Programa principal
404 # ============================================================================
405
406 def main() -> None:
407     parser = argparse.ArgumentParser(description="SOM linear de Kohonen - Fausett Ex. 4.1")
408     parser.add_argument("--seed", type=int, default=SEED_PADRAO,
409                         help="seed da rede (pesos iniciais e ordem de apresentacao)")
410     parser.add_argument("--seed-dados", type=int, default=SEED_DADOS,
411                         help="seed do gerador dos dados de treino")
```

```
    parser.add_argument("--decaimento", choices=list(DECAIMENTOS), default="linear")
    parser.add_argument("--saida", default=".", help="pasta de saida")
    parser.add_argument("--sem-extras", action="store_true",
                        help="nao roda as comparacoes extras (decaimento/seeds)")
    args = parser.parse_args()

    pasta_fig = os.path.join(args.saida, "figuras")
    os.makedirs(pasta_fig, exist_ok=True)

    # --- dados: seed PROPRIA e independente da seed da rede. Assim (i) o mesmo
    # conjunto serve a todos os experimentos e (ii) os pesos iniciais nao sao
    # correlacionados com os pontos (usar o mesmo gerador nos dois faria os
    # primeiros pesos serem copias escaladas dos primeiros pontos sorteados).
    rng_dados = np.random.default_rng(args.seed_dados)
    dados = gerar_dados_disco(N_PONTOS, rng_dados)
    caminho_csv = os.path.join(args.saida, "dados_treino.csv")
    salvar_csv(caminho_csv, dados)
    dados = carregar_csv(caminho_csv) # garante que o treino usa o arquivo
    print(f"[dados] salvos em {caminho_csv}\n")

    # --- experimento principal
    print(f"=== Experimento principal: decaimento {args.decaimento}, seed {args.seed} ===")
    principal = treinar(dados, seed=args.seed, decaimento=DECAIMENTOS[args.decaimento],
                        epocas_registro=EPOCAS_PLOT)
    som = principal["som"]
    print(f"\nFinal: EQ = {principal['erros'][-1]:.5f} | comprimento da cadeia = "
          f"{som.comprimento_cadeia():.3f} | cruzamentos = {som.n_cruzamentos()} | "
          f"unidades mortas = {som.unidades_mortas(dados)}")
    np.savetxt(os.path.join(args.saida, "pesos_finais.csv"), som.pesos, delimiter=",",
               header="w1j,w2j", comments="", fmt="%.10f")

    salvar_figuras_epocas(principal["snapshots"], dados, pasta_fig,
                          principal["alfas"], principal["erros"])
    salvar_painel(principal["snapshots"], dados,
                  os.path.join(pasta_fig, "som_painel.png"), principal["erros"])

    alfas = {args.decaimento: principal["alfas"]}
    erros = {args.decaimento: principal["erros"]}

    # --- analises extras
    if not args.sem_extras:
        outro = "exponencial" if args.decaimento == "linear" else "linear"
        print(f"\n=== Extra: decaimento {outro}, seed {args.seed} ===")
        res_outro = treinar(dados, seed=args.seed, decaimento=DECAIMENTOS[outro])
        alfas[outro] = res_outro["alfas"]
        erros[outro] = res_outro["erros"]
        salvar_comparacao({f"decaimento {args.decaimento}": principal,
                           f"decaimento {outro}": res_outro}, dados,
                          os.path.join(pasta_fig, "comparacao_decaimento.png"),
                          "Configuração final (época 100): decaimento linear \u00d7 exponencial")

        print("\n=== Extra: seeds diferentes (pesos iniciais e ordem), mesmos dados ===")
        seeds = [1, 7, 2024, 31415]
        res_seeds = {}
        for s in seeds:
            r = treinar(dados, seed=s, decaimento=DECAIMENTOS[args.decaimento],
                        verbose=False)
            res_seeds[f"seed {s}"] = r
            print(f"seed {s:>6}: EQ = {r['erros'][-1]:.5f} | cruzamentos = "
```

```
                f"{r['som'].n_cruzamentos()} | mortas = "
                f"{r['som'].unidades_mortas(dados)} | comprimento = "
                f"{r['som'].comprimento_cadeia():.3f}")
        salvar_comparacao(res_seeds, dados,
                          os.path.join(pasta_fig, "comparacao_seeds.png"),
                          f"Configuração final para seeds diferentes "
                          f"(decaimento {args.decaimento})")

        print("\n=== Extra (fora do enunciado): raio decrescente 10 -> 1 ===")
        res_raio = treinar(dados, seed=args.seed, decaimento=DECAIMENTOS[args.decaimento],
                           raio_fn=raio_decrescente)
        salvar_comparacao({"R = 1 fixo (enunciado)": principal,
                           "R: 10 \u2192 1 (épocas 1\u201350)": res_raio}, dados,
                          os.path.join(pasta_fig, "comparacao_raio.png"),
                          "Efeito do raio de vizinhança (mesma seed e mesmos dados)")

    salvar_grafico_alfa(alfas, os.path.join(pasta_fig, "alpha_por_epoca.png"))
    salvar_grafico_erro(erros, os.path.join(pasta_fig, "erro_quantizacao.png"))
    print(f"\nFiguras salvas em {pasta_fig}/")


if __name__ == "__main__":
    main()
```